C-DAC Mumbai DBT –Day04

Relational Algebra and Its Mapping to SQL

Relational algebra is a procedural query language used in relational databases. It provides operations to retrieve and combine data from tables.

In simple words, relational algebra describes operations that we can perform on relations (tables). Many of these operations have corresponding SQL queries.

1. Selection (σ) — Selecting rows

The selection operation retrieves rows that satisfy a given condition.

- Relational algebra symbol: **σ** (sigma)
- SQL equivalent: WHERE

SELECT *

FROM Student

WHERE city = 'Pune';

Projection (π) — Selecting columns

The projection operation retrieves specific columns from a table.

- Relational algebra symbol: **π** (pi)
- SQL equivalent: SELECT column list

Relational algebra expression:

Union (∪) — Combining rows

The union operation combines rows from two compatible tables or relations.

- Relational algebra symbol: ∪
- SQL equivalent: UNION

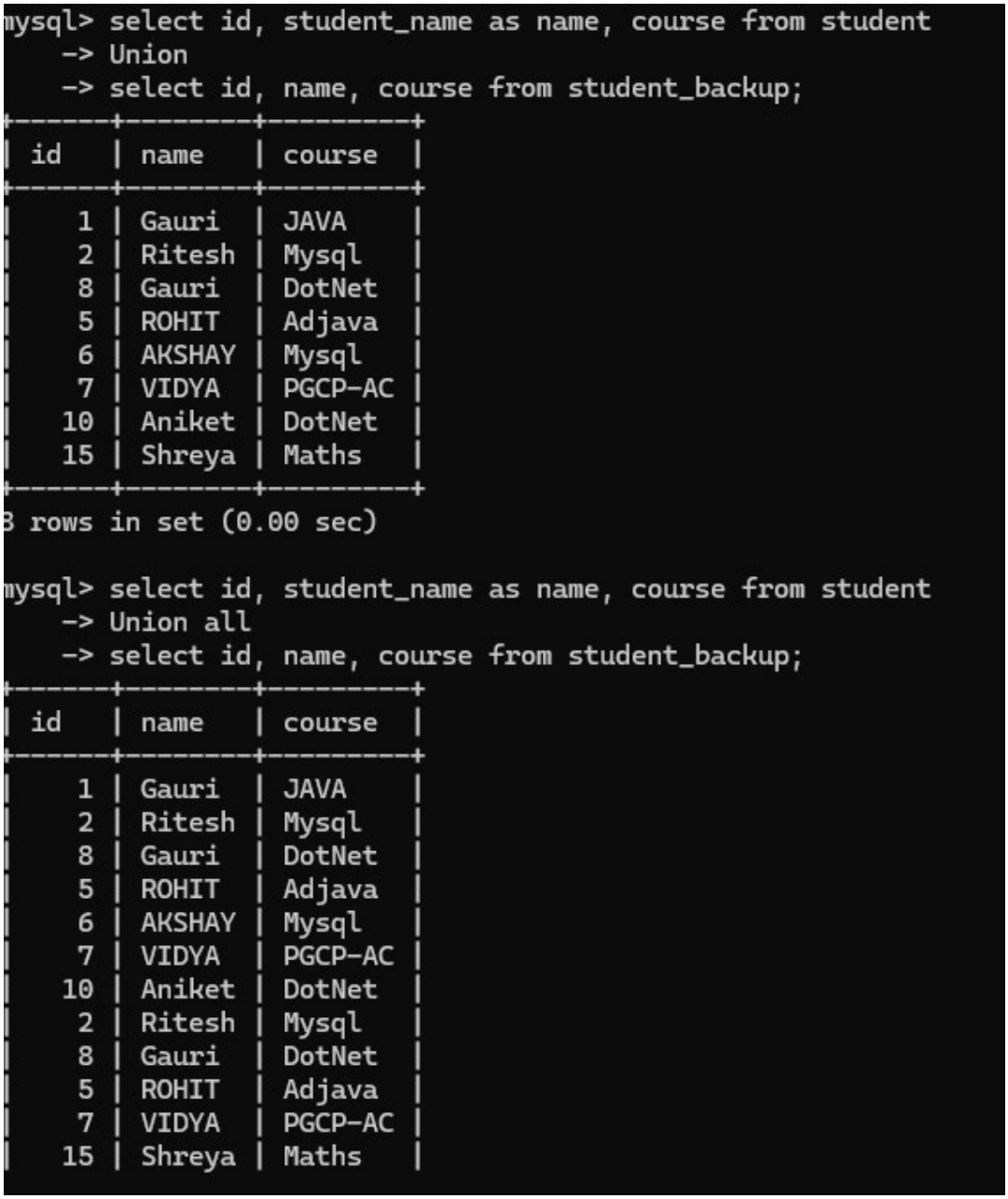
Important: The two queries must return the same number of columns, with compatible data types in corresponding positions.

- UNION removes duplicate rows.

- UNION ALL keeps duplicate rows.

select id, name, course from student_backup
union
select id, student_name, course from student

select id, name, course from student_backup
Union all
select id, student_name, course from student



```
mysql> select id, student_name as name, course from student
    -> Union
    -> select id, name, course from student_backup;
+------+--------+---------+
| id   | name   | course  |
+------+--------+---------+
|    1 | Gauri  | JAVA    |
|    2 | Ritesh | Mysql   |
|    8 | Gauri  | DotNet  |
|    5 | ROHIT  | Adjava  |
|    6 | AKSHAY | Mysql   |
|    7 | VIDYA  | PGCP-AC |
|   10 | Aniket | DotNet  |
|   15 | Shreya | Maths   |
+------+--------+---------+
8 rows in set (0.00 sec)

mysql> select id, student_name as name, course from student
    -> Union all
    -> select id, name, course from student_backup;
+------+--------+---------+
| id   | name   | course  |
+------+--------+---------+
|    1 | Gauri  | JAVA    |
|    2 | Ritesh | Mysql   |
|    8 | Gauri  | DotNet  |
|    5 | ROHIT  | Adjava  |
|    6 | AKSHAY | Mysql   |
|    7 | VIDYA  | PGCP-AC |
|   10 | Aniket | DotNet  |
|    2 | Ritesh | Mysql   |
|    8 | Gauri  | DotNet  |
|    5 | ROHIT  | Adjava  |
|    7 | VIDYA  | PGCP-AC |
|   15 | Shreya | Maths   |
```

Intersection (∩) — Finding common rows

The intersection operation returns rows that are present in both relations.

- Relational algebra symbol: ∩
- SQL equivalent: INTERSECT in MySQL 8.0.31 and later.

Relational algebra expression:

```
select id, name, course from student_backup
intersect
select id, student_name, course from student;
```

Difference (−) — Finding rows in one relation but not the other

The difference operation returns rows present in the first relation but absent from the second.

- Relational algebra symbol: −
- SQL equivalent: EXCEPT in MySQL 8.0.31 and later.

Relational algebra expression:

```
select id, student_name, course from student
except
select id, name, course from student_backup
```

```
mysql> select id, name, course from student_backup
    -> intersect
    -> select id, student_name, course from student;
+------+--------+---------+
| id   | name   | course  |
+------+--------+---------+
|    2 | Ritesh | Mysql   |
|    8 | Gauri  | DotNet  |
|    5 | ROHIT  | Adjava  |
|    7 | VIDYA  | PGCP-AC |
+------+--------+---------+
4 rows in set (0.00 sec)

mysql> select id, student_name, course from student
    -> except
    -> select id, name, course from student_backup;
+------+--------------+--------+
| id   | student_name | course |
+------+--------------+--------+
|    1 | Gauri        | JAVA   |
|    6 | AKSHAY       | Mysql  |
|   10 | Aniket       | DotNet |
+------+--------------+--------+
```

1. What is a JOIN?

A JOIN combines rows from two or more tables using a related column or a specified condition. In this example, the student table stores student details and dept_id connects a student to the department table.

The examples follow this sequence: CROSS JOIN → INNER JOIN / EQUI JOIN → old comma syntax → USING → NATURAL JOIN → NON-EQUI JOIN → LEFT JOIN → RIGHT JOIN → FULL OUTER JOIN workaround → SELF JOIN → three-table JOIN → JOIN with GROUP BY.

2. Create and populate the example tables

```
CREATE DATABASE IF NOT EXISTS college_demo;
USE college_demo;

DROP TABLE IF EXISTS student_course;
DROP TABLE IF EXISTS student;
DROP TABLE IF EXISTS course;
DROP TABLE IF EXISTS department;
DROP TABLE IF EXISTS dept_id_band;
```

```
CREATE TABLE department (
    dept_id INT PRIMARY KEY,
    dept_name VARCHAR(40) NOT NULL
);

INSERT INTO department VALUES
(10, 'Computer'),
(20, 'IT'),
(30, 'Electronics'),
(40, 'Mechanical');

CREATE TABLE student (
    student_id INT PRIMARY KEY,
    name VARCHAR(40) NOT NULL,
    dept_id INT NULL,
    mentor_id INT NULL,
    CONSTRAINT fk_student_department
    FOREIGN KEY (dept_id) REFERENCES department(dept_id),
    CONSTRAINT fk_student_mentor
    FOREIGN KEY (mentor_id) REFERENCES student(student_id)
);

INSERT INTO student VALUES
(1, 'Rahul', 10, NULL),
(2, 'Priya', 20, 1),
(3, 'Amit', 10, 1),
(4, 'Sneha', NULL, 2),
(5, 'Karan', 30, 1),
(6, 'Neha', 20, 2),
(7, 'Vikram', 10, 3),
(8, 'Pooja', 30, 5),
(9, 'Rohan', 20, 2),
(10, 'Isha', NULL, 1),
(11, 'Arjun', 10, 7),
(12, 'Meera', 30, 5);

CREATE TABLE course (
    course_id INT PRIMARY KEY,
    course_name VARCHAR(50) NOT NULL
);

INSERT INTO course VALUES
(101, 'Java'),
```

```
(102, 'MySQL'),
(103, 'Python'),
(104, 'Web Development'),
(105, 'Data Structures'),
(106, 'Cloud Computing');

CREATE TABLE student_course (
    student_id INT,
    course_id INT,
    PRIMARY KEY (student_id, course_id),
    FOREIGN KEY (student_id) REFERENCES student(student_id),
    FOREIGN KEY (course_id) REFERENCES course(course_id)
);

INSERT INTO student_course VALUES
(1,101),(1,102),(2,102),(2,104),(3,101),
(3,103),(4,104),(5,103),(5,105),(6,102),
(7,101),(7,105),(8,103),(9,104),(11,105);
```

The sample data intentionally includes students with no department (Sneha and Isha), students with no course enrollment (for example, Meera), and a course with no enrollment . These cases make outer joins easier to understand.

3. CROSS JOIN

3.1 CROSS JOIN

A CROSS JOIN combines every row from the first table with every row from the second table.

```
SELECT s.name, d.dept_name
 FROM student AS s
 CROSS JOIN department AS d;
```

Theory

- **Purpose:** To produce every possible combination of rows from two tables.
- **Condition:** No matching condition is required.
- **Result:** 12 students × 4 departments = 48 rows.
- **Common use:** Useful for generating combinations, such as every student paired with every department.
- **Caution:** The result can become very large when either table contains many rows.

```
mysql> select s.name, d.dept_name
    ->  from student as s
    ->  cross join department as d;
+--------+-------------+
| name   | dept_name   |
+--------+-------------+
| Rahul  | Mechanical  |
| Rahul  | Electronics |
| Rahul  | IT          |
| Rahul  | Computer    |
| Priya  | Mechanical  |
| Priya  | Electronics |
| Priya  | IT          |
| Priya  | Computer    |
| Amit   | Mechanical  |
| Amit   | Electronics |
| Amit   | IT          |
| Amit   | Computer    |
| Sneha  | Mechanical  |
| Sneha  | Electronics |
| Sneha  | IT          |
| Sneha  | Computer    |
| Karan  | Mechanical  |
| Karan  | Electronics |
| Karan  | IT          |
| Karan  | Computer    |
| Neha   | Mechanical  |
| Neha   | Electronics |
| Neha   | IT          |
| Neha   | Computer    |
| Vikram | Mechanical  |
| Vikram | Electronics |
| Vikram | IT          |
```

4. INNER JOIN and EQUI JOIN

4.1 INNER JOIN

Returns only rows for which the join condition finds a match in both tables.

```
SELECT s.student_id, s.name, d.dept_name
 FROM student AS s
 INNER JOIN department AS d
      ON s.dept_id = d.dept_id;
```

Theory

- **Purpose:** To show students who are assigned to a department.
- **Condition:** The ON clause states how rows from the two tables are related.

- **Result:** Only matching student–department rows appear. Students whose dept_id is NULL are excluded.
- **Common use:** Use it when the output should contain only records that have a match in both tables.
- **Database design:** Related details are stored in separate tables and combined when needed, reducing repeated department names in the student table.

```
mysql> select student.student_id, student.name , department.dept_name
    ->  from student
    ->  inner join department
    ->  On student.dept_id=department.dept_id;
+------------+--------+-------------+
| student_id | name   | dept_name   |
+------------+--------+-------------+
|          1 | Rahul  | Computer    |
|          3 | Amit   | Computer    |
|          7 | Vikram | Computer    |
|         11 | Arjun  | Computer    |
|          2 | Priya  | IT          |
|          6 | Neha   | IT          |
|          9 | Rohan  | IT          |
|          5 | Karan  | Electronics |
|          8 | Pooja  | Electronics |
|         12 | Meera  | Electronics |
+------------+--------+-------------+
```

4.2 EQUI JOIN

An equi join is a join in which the matching condition uses the equality operator (=). It is a type of join condition, not a separate JOIN keyword.

```
SELECT s.name, d.dept_name
 FROM student AS s
 JOIN department AS d
      ON s.dept_id = d.dept_id;
```

Theory

- **Purpose:** To match rows where the values in the related columns are equal.
- **Equality condition:** The ON condition uses =, here s.dept_id = d.dept_id.
- **Result:** Only matching rows are returned because this example uses INNER JOIN.
- **Important distinction:** INNER JOIN describes which rows are returned; EQUI JOIN describes the equality-based condition.
- **Common use:** Very common for matching primary-key and foreign-key values.

5. Old comma syntax with WHERE

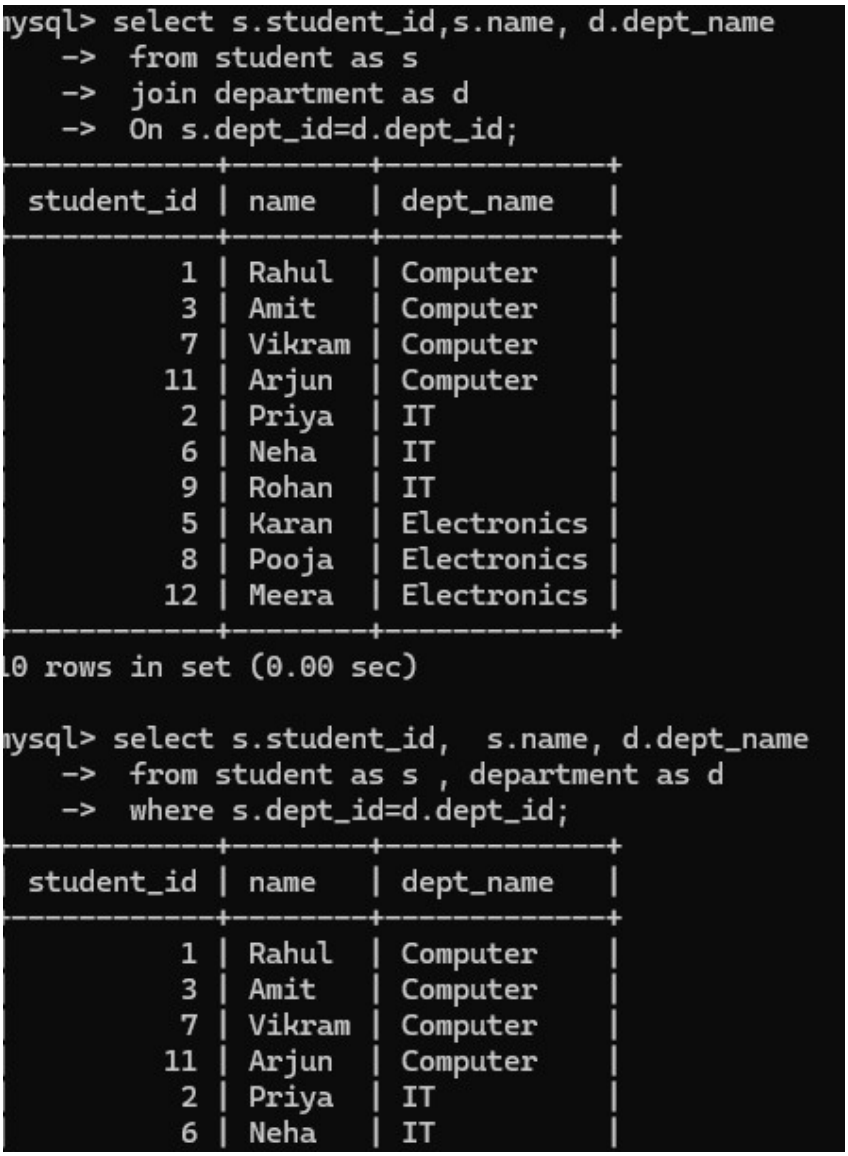
5.1 Comma join (older style)

This is an older way to write an inner/equi join. Both tables are listed in FROM, and the matching condition is placed in WHERE.

```
SELECT s.name, d.dept_name
 FROM student AS s, department AS d
 WHERE s.dept_id = d.dept_id;
```

Theory

- **Purpose:** To retrieve matching rows from two tables using older SQL syntax.
- **Condition:** The equality condition is written in WHERE.
- **Result:** For this example, the result is equivalent to the INNER JOIN query above.
- **Recommendation:** Prefer explicit JOIN ... ON syntax because it separates join conditions from ordinary filters and is easier to read.
- **Caution:** If the join condition is accidentally omitted, the query can produce a Cartesian product.



```
mysql> select s.student_id,s.name, d.dept_name
    ->  from student as s
    ->  join department as d
    ->  On s.dept_id=d.dept_id;
+------------+--------+-------------+
| student_id | name   | dept_name   |
+------------+--------+-------------+
|          1 | Rahul  | Computer    |
|          3 | Amit   | Computer    |
|          7 | Vikram | Computer    |
|         11 | Arjun  | Computer    |
|          2 | Priya  | IT          |
|          6 | Neha   | IT          |
|          9 | Rohan  | IT          |
|          5 | Karan  | Electronics |
|          8 | Pooja  | Electronics |
|         12 | Meera  | Electronics |
+------------+--------+-------------+
10 rows in set (0.00 sec)

mysql> select s.student_id,  s.name, d.dept_name
    ->  from student as s , department as d
    ->  where s.dept_id=d.dept_id;
+------------+--------+-------------+
| student_id | name   | dept_name   |
+------------+--------+-------------+
|          1 | Rahul  | Computer    |
|          3 | Amit   | Computer    |
|          7 | Vikram | Computer    |
|         11 | Arjun  | Computer    |
|          2 | Priya  | IT          |
|          6 | Neha   | IT          |
```

6. JOIN ... USING

6.1 USING

USING is convenient when both tables have a join column with exactly the same name.

```
SELECT s.name, d.dept_name
 FROM student AS s
 JOIN department AS d
 USING (dept_id);
```

Theory

- **Purpose:** To write a simpler join when the shared column has the same name in both tables.
- **Condition:** USING (dept_id) means the rows are matched by equal dept_id values.

- **Result:** Only matching rows are returned in this INNER JOIN example.
- **Difference from ON:** ON can compare columns with different names and can express more complex conditions; USING names the shared column directly.
- **Common use:** Useful for straightforward joins on identically named columns.

7. NATURAL JOIN

7.1 NATURAL JOIN

NATURAL JOIN automatically matches columns that have the same name in both tables.

```
SELECT s.name, d.dept_name
 FROM student AS s
 NATURAL JOIN department AS d;
```

Theory

- **Purpose:** To join tables automatically through all identically named columns.
- **Condition:** No ON or USING clause is written; MySQL identifies the common column names.
- **Result:** Here the shared column is dept_id, so matching is based on that column.
- **Difference from EQUI JOIN:** NATURAL JOIN is not a synonym for EQUI JOIN. EQUI JOIN explicitly uses =; NATURAL JOIN automatically chooses common column names.
- **Caution:** If another same-named column is added to both tables later, the NATURAL JOIN may use it too and change the results. Explicit JOIN ... ON is often clearer.

```
mysql> select s.student_id, s.name, d.dept_name
    ->  from student s
    ->  join department d
    ->  Using (dept_id);
+------------+--------+-------------+
| student_id | name   | dept_name   |
+------------+--------+-------------+
|          1 | Rahul  | Computer    |
|          3 | Amit   | Computer    |
|          7 | Vikram | Computer    |
|         11 | Arjun  | Computer    |
|          2 | Priya  | IT          |
|          6 | Neha   | IT          |
|          9 | Rohan  | IT          |
|          5 | Karan  | Electronics |
|          8 | Pooja  | Electronics |
|         12 | Meera  | Electronics |
+------------+--------+-------------+
10 rows in set (0.00 sec)

mysql> Select s.student_id, s.name, d.dept_id
    ->  from student as s
    ->  natural join department as d;
+------------+--------+---------+
| student_id | name   | dept_id |
+------------+--------+---------+
|          1 | Rahul  |      10 |
|          3 | Amit   |      10 |
|          7 | Vikram |      10 |
|         11 | Arjun  |      10 |
|          2 | Priya  |      20 |
|          6 | Neha   |      20 |
|          9 | Rohan  |      20 |
|          5 | Karan  |      30 |
```

8. NON-EQUI JOIN

A non-equi join uses a condition other than equality, such as <, >, <=, >= or BETWEEN. The following helper table assigns a department-ID band to each label.

8.1 NON-EQUI JOIN

The join condition compares a value with a range instead of matching two columns using equality.

Theory

- **Purpose:** To match rows using a range or another comparison rather than simple equality.
- **Condition:** This example uses BETWEEN with >= and <= range logic.
- **Result:** Each department is matched to the band that contains its dept_id.
- **Common use:** Useful for salary bands, age ranges, grading scales, tax slabs, or date ranges.
- **Important point:** A non-equi join can still be an INNER JOIN; "non-equi" describes the condition, not which unmatched rows are preserved.

```
mysql> select s.student_id, s.name, s.dept_id as my_dept_id , d.dept_id
    ->  from student as s
    ->  join department as d
    ->  On s.dept_id > d.dept_id;
+------------+-------+------------+---------+
| student_id | name  | my_dept_id | dept_id |
+------------+-------+------------+---------+
|          2 | Priya |         20 |      10 |
|          6 | Neha  |         20 |      10 |
|          9 | Rohan |         20 |      10 |
|          5 | Karan |         30 |      10 |
|          8 | Pooja |         30 |      10 |
|         12 | Meera |         30 |      10 |
|          5 | Karan |         30 |      20 |
|          8 | Pooja |         30 |      20 |
|         12 | Meera |         30 |      20 |
+------------+-------+------------+---------+
9 rows in set (0.00 sec)
```

9. LEFT JOIN

9.1 LEFT JOIN

Returns every row from the left table and matching rows from the right table. If no match exists, right-table columns contain NULL.

```
SELECT s.student_id, s.name, d.dept_name
 FROM student AS s
 LEFT JOIN department AS d
       ON s.dept_id = d.dept_id
 ORDER BY s.student_id;
```

Theory

- **Purpose:** To keep every student in the output, even if the student has no department.
- **Condition:** Matching is done using s.dept_id = d.dept_id.
- **Result:** All 12 students appear. Sneha and Isha have NULL for dept_name because they have no department match.

- **Common use:** Useful for finding missing relationships, such as students who have not been assigned to a department.
- **Remember:** The table written before LEFT JOIN is the left table and all its rows are preserved.

```
mysql> select s.student_id, s.name, d.dept_name
    ->  from student s
    ->  left join department d
    ->  On s.dept_id=d.dept_id
    ->  order by s.student_id;
+------------+--------+-------------+
| student_id | name   | dept_name   |
+------------+--------+-------------+
|          1 | Rahul  | Computer    |
|          2 | Priya  | IT          |
|          3 | Amit   | Computer    |
|          4 | Sneha  | NULL        |
|          5 | Karan  | Electronics |
|          6 | Neha   | IT          |
|          7 | Vikram | Computer    |
|          8 | Pooja  | Electronics |
|          9 | Rohan  | IT          |
|         10 | Isha   | NULL        |
|         11 | Arjun  | Computer    |
|         12 | Meera  | Electronics |
+------------+--------+-------------+
12 rows in set (0.00 sec)
```

10. RIGHT JOIN

10.1 RIGHT JOIN

Returns every row from the right table and matching rows from the left table. If no match exists, left-table columns contain NULL.

```
SELECT s.name, d.dept_id, d.dept_name
 FROM student AS s
 RIGHT JOIN department AS d
      ON s.dept_id = d.dept_id
 ORDER BY d.dept_id, s.student_id;
```

Theory

- **Purpose:** To keep every department in the output, including departments with no students.
- **Condition:** The ON clause matches student.dept_id with department.dept_id.

- **Result:** All departments are shown. If a department has no student, the student columns would be NULL.
- **Common use:** Useful when the right-side table is the main list that must be preserved.
- **Remember:** RIGHT JOIN preserves all rows from the table written after RIGHT JOIN. The same logic can often be written as a LEFT JOIN by reversing table order.

```
mysql> select s.student_id, s.name, d.dept_name
    ->  from student s
    ->  right join department d
    ->  On s.dept_id=d.dept_id
    ->  order by s.student_id;
+------------+--------+-------------+
| student_id | name   | dept_name   |
+------------+--------+-------------+
|       NULL | NULL   | Mechanical  |
|          1 | Rahul  | Computer    |
|          2 | Priya  | IT          |
|          3 | Amit   | Computer    |
|          5 | Karan  | Electronics |
|          6 | Neha   | IT          |
|          7 | Vikram | Computer    |
|          8 | Pooja  | Electronics |
|          9 | Rohan  | IT          |
|         11 | Arjun  | Computer    |
|         12 | Meera  | Electronics |
+------------+--------+-------------+
```

11. FULL OUTER JOIN in MySQL

11.1 FULL OUTER JOIN workaround

MySQL does not support FULL OUTER JOIN directly. A common workaround combines LEFT JOIN and RIGHT JOIN results with UNION.

```
SELECT s.student_id, s.name, d.dept_name
 FROM student AS s
 LEFT JOIN department AS d
      ON s.dept_id = d.dept_id

 UNION

 SELECT s.student_id, s.name, d.dept_name
 FROM student AS s
```

```
RIGHT JOIN department AS d
    ON s.dept_id = d.dept_id;
```

Theory

- **Purpose:** To return matching rows plus unmatched rows from both tables.
- **Condition:** Both parts use the same matching condition.
- **Result:** Students without a department and departments without a student can both be included.
- **How UNION helps:** UNION combines the results and removes duplicate rows.
- **Alternative:** UNION ALL can be used with an explicit filter to avoid duplicating matched rows; for a beginner example, UNION is the simpler workaround.
- **Important point:** This is an emulation of FULL OUTER JOIN, not native MySQL FULL OUTER JOIN syntax.

```
mysql> select s.student_id, s.name, d.dept_name
    ->  from student as s
    ->  left join department as d
    ->  On s.dept_id=d.dept_id
    ->  Union
    ->  select s.student_id, s.name, d.dept_name
    ->  from student as s
    ->  right join department as d
    ->  On s.dept_id=d.dept_id;
+------------+--------+-------------+
| student_id | name   | dept_name   |
+------------+--------+-------------+
|          1 | Rahul  | Computer    |
|          2 | Priya  | IT          |
|          3 | Amit   | Computer    |
|          4 | Sneha  | NULL        |
|          5 | Karan  | Electronics |
|          6 | Neha   | IT          |
|          7 | Vikram | Computer    |
|          8 | Pooja  | Electronics |
|          9 | Rohan  | IT          |
|         10 | Isha   | NULL        |
|         11 | Arjun  | Computer    |
|         12 | Meera  | Electronics |
|       NULL | NULL   | Mechanical  |
+------------+--------+-------------+
13 rows in set (0.00 sec)
```

12. SELF JOIN

12.1 SELF JOIN

A self join joins a table to itself. Aliases are required to distinguish the two uses of the same table.

```
SELECT s.name AS student_name,
       m.name AS mentor_name
FROM student AS s
LEFT JOIN student AS m
       ON s.mentor_id = m.student_id
ORDER BY s.student_id;
```

Theory

- **Purpose:** To compare or connect records within the same table.
- **Condition:** The student's mentor_id matches another student's student_id.
- **Result:** The output shows each student and their mentor. Students without a mentor still appear because LEFT JOIN is used.
- **Common use:** Useful for employee–manager, student–mentor, category–parent category, or person–supervisor relationships.
- **Important point:** Aliases such as s and m make it clear which role each copy of the table represents.

```
mysql> select s.name as student_namem,
    ->  m.name as mentor_name
    ->  from student as s
    ->  left join student as m
    ->  On s.mentor_id=m.student_id
    ->  order by s.student_id;
+---------------+-------------+
| student_namem | mentor_name |
+---------------+-------------+
| Rahul         | NULL        |
| Priya         | Rahul       |
| Amit          | Rahul       |
| Sneha         | Priya       |
| Karan         | Rahul       |
| Neha          | Priya       |
| Vikram        | Amit        |
| Pooja         | Karan       |
| Rohan         | Priya       |
| Isha          | Rahul       |
| Arjun         | Vikram      |
| Meera         | Karan       |
+---------------+-------------+
12 rows in set (0.00 sec)
```

13. JOIN across three tables

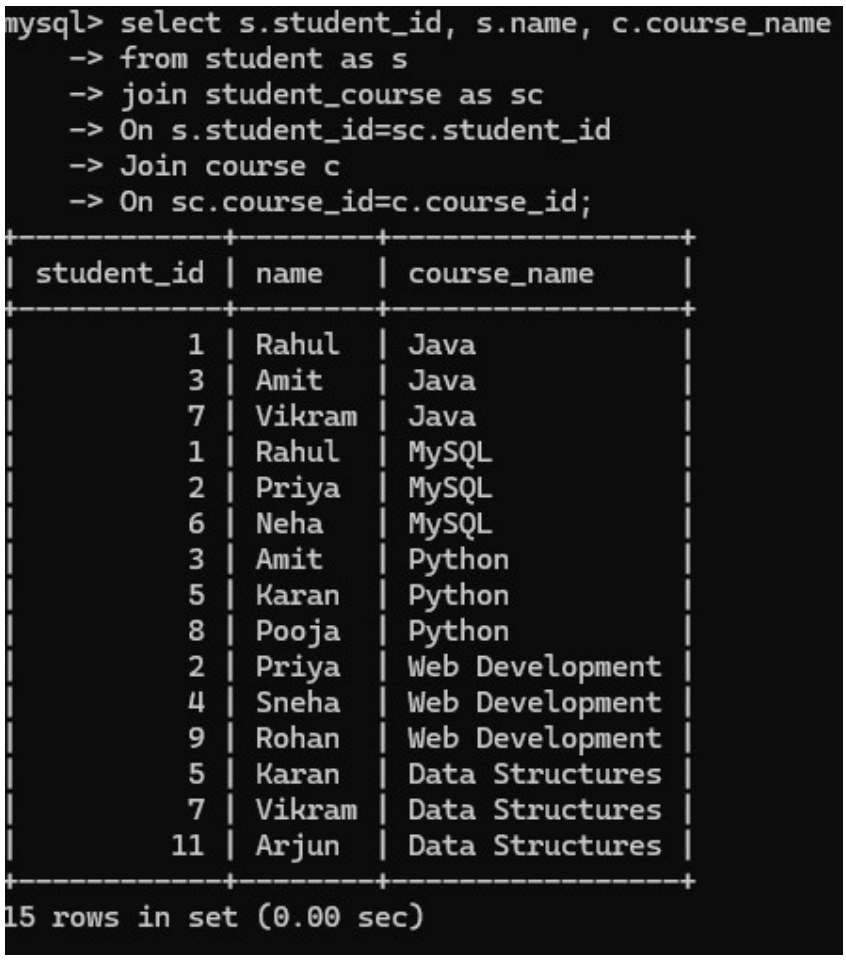
13.1 Three-table JOIN

Joins can combine more than two tables when the data is connected through related columns.

```
SELECT s.name, c.course_name
 FROM student AS s
 JOIN student_course AS sc
      ON s.student_id = sc.student_id
 JOIN course AS c
      ON sc.course_id = c.course_id
 ORDER BY s.student_id, c.course_id;
```

Theory

- **Purpose:** To display student names along with the courses in which they are enrolled.
- **Condition:** student connects to student_course through student_id; student_course connects to course through course_id.
- **Result:** Each output row represents one student-course enrollment.
- **Why a linking table is used:** A student can enroll in many courses and a course can have many students. student_course represents this many-to-many relationship.
- **Common use:** Useful when information is spread across several related tables.



```
mysql> select s.student_id, s.name, c.course_name
    -> from student as s
    -> join student_course as sc
    -> On s.student_id=sc.student_id
    -> Join course c
    -> On sc.course_id=c.course_id;
+------------+--------+-----------------+
| student_id | name   | course_name     |
+------------+--------+-----------------+
|          1 | Rahul  | Java            |
|          3 | Amit   | Java            |
|          7 | Vikram | Java            |
|          1 | Rahul  | MySQL           |
|          2 | Priya  | MySQL           |
|          6 | Neha   | MySQL           |
|          3 | Amit   | Python          |
|          5 | Karan  | Python          |
|          8 | Pooja  | Python          |
|          2 | Priya  | Web Development |
|          4 | Sneha  | Web Development |
|          9 | Rohan  | Web Development |
|          5 | Karan  | Data Structures |
|          7 | Vikram | Data Structures |
|         11 | Arjun  | Data Structures |
+------------+--------+-----------------+
15 rows in set (0.00 sec)
```

14. JOIN with GROUP BY

14.1 Count enrollments for every course

LEFT JOIN keeps courses even if nobody has enrolled in them.

```
SELECT c.course_name,
       COUNT(sc.student_id) AS total_enrollments
FROM course AS c
LEFT JOIN student_course AS sc
       ON c.course_id = sc.course_id
GROUP BY c.course_id, c.course_name
ORDER BY c.course_id;
```

Theory

- **Purpose:** To calculate the number of enrollments for each course.
- **Condition:** Courses are matched to enrollment rows using course_id.
- **Result:** Every course appears, including Cloud Computing with a count of 0.
- **Why COUNT(sc.student_id):** COUNT(column) ignores NULL values, so a course with no matching enrollment gets 0. COUNT(*) would count the outer-join placeholder row and could show 1 instead.
- **Common use:** Useful for counts, totals, averages, and other summaries across related tables.

15. Quick comparison

Join / pattern	Rows returned	Main idea
CROSS JOIN	Every combination	No matching condition
INNER JOIN	Matching rows only	Match rows using ON/USING
EQUI JOIN	Depends on join type; commonly matches only	Condition uses =
NON-EQUI JOIN	Rows satisfying a comparison/range	Condition uses <, >, BETWEEN, etc.
LEFT JOIN	All left rows + matches	Unmatched right values become NULL
RIGHT JOIN	All right rows + matches	Unmatched left values become NULL
FULL OUTER workaround	Matches + unmatched rows from both sides	In MySQL, combine outer joins with UNION
SELF JOIN	Depends on join type	Same table used twice with aliases

USING	Depends on join type	Shared same-named join column
NATURAL JOIN	Matching rows by all common column names	Automatic join condition
JOIN + GROUP BY	Summary rows	Aggregate joined data

MySQL Window Functions and CTEs

Window functions

1. GROUP BY limitation

GROUP BY creates one result row per group (or group combination). It is ideal for summaries, but it does not preserve each employee row in the grouped result.

```
SELECT department, AVG(salary) AS avg_salary
FROM employee
GROUP BY department;
```

Theory

- The result has one row per department, not one row per employee.
- To show every employee along with a department-level average, use a window function.
- GROUP BY can include multiple grouping columns, but the result is still at the grouping level.

```
mysql> select department, sum(salary) from employee
    -> group by department;
+------------+-------------+
| department | sum(salary) |
+------------+-------------+
| IT         |   260000.00 |
| HR         |   160000.00 |
| Sales      |   155000.00 |
+------------+-------------+
```

2. OVER() with no options

An empty OVER() uses all rows in the query's result set as one window.

```
SELECT emp_name, salary,
    AVG(salary) OVER () AS overall_avg_salary
FROM employee;
```

Theory

- Calculates one overall average salary.
- Displays that average beside every employee row.
- Unlike GROUP BY, it preserves individual rows.

3. OVER(PARTITION BY ...)

PARTITION BY divides rows into separate windows for calculation.

```
SELECT emp_name, department, salary,
      AVG(salary) OVER (
         PARTITION BY department
      ) AS dept_avg_salary
FROM employee;
```

Theory

- Each department forms a separate partition.
- The average is calculated independently within each department.
- All employee rows remain in the output.

```
mysql> select emp_name, salary,
    ->  Avg(salary) over() as avg_salary
    -> from employee;
+----------+----------+--------------+
| emp_name | salary   | avg_salary   |
+----------+----------+--------------+
| Amit     | 60000.00 | 57500.000000 |
| Priya    | 80000.00 | 57500.000000 |
| Rahul    | 80000.00 | 57500.000000 |
| Sneha    | 50000.00 | 57500.000000 |
| Neha     | 60000.00 | 57500.000000 |
| Karan    | 45000.00 | 57500.000000 |
| Pooja    | 55000.00 | 57500.000000 |
| Vikram   | 55000.00 | 57500.000000 |
| Meera    | 40000.00 | 57500.000000 |
| Arjun    | 50000.00 | 57500.000000 |
+----------+----------+--------------+
10 rows in set (0.00 sec)

mysql>
mysql>  select emp_name, salary,department,
    ->  avg(salary) over( partition by department) as dept_avg_salary
    ->  from employee;
+----------+----------+------------+-----------------+
| emp_name | salary   | department | dept_avg_salary |
+----------+----------+------------+-----------------+
| Sneha    | 50000.00 | HR         |    53333.333333 |
| Neha     | 60000.00 | HR         |    53333.333333 |
| Arjun    | 50000.00 | HR         |    53333.333333 |
| Amit     | 60000.00 | IT         |    65000.000000 |
| Priya    | 80000.00 | IT         |    65000.000000 |
| Rahul    | 80000.00 | IT         |    65000.000000 |
| Meera    | 40000.00 | IT         |    65000.000000 |
| Karan    | 45000.00 | Sales      |    51666.666667 |
```

4. OVER(ORDER BY ...)

ORDER BY inside OVER defines the order used by the window calculation. It does not guarantee final display order.

```
SELECT emp_name, salary,
    SUM(salary) OVER (
       ORDER BY salary, emp_id
       ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
    ) AS running_total
FROM employee
ORDER BY salary, emp_id;
```

Theory

- The window calculation processes rows in salary order, using emp_id to break ties.
- The final ORDER BY controls how the output is displayed.
- An explicit frame avoids ambiguity when calculating running totals.

```
mysql> select emp_name, salary,department,
    ->  avg(salary) over( order by salary, emp_id ) as dept_avg_salary
    ->  from employee;
+----------+----------+------------+-----------------+
| emp_name | salary   | department | dept_avg_salary |
+----------+----------+------------+-----------------+
| Meera    | 40000.00 | IT         |    40000.000000 |
| Karan    | 45000.00 | Sales      |    42500.000000 |
| Sneha    | 50000.00 | HR         |    45000.000000 |
| Arjun    | 50000.00 | HR         |    46250.000000 |
| Pooja    | 55000.00 | Sales      |    48000.000000 |
| Vikram   | 55000.00 | Sales      |    49166.666667 |
| Amit     | 60000.00 | IT         |    50714.285714 |
| Neha     | 60000.00 | HR         |    51875.000000 |
| Priya    | 80000.00 | IT         |    55000.000000 |
| Rahul    | 80000.00 | IT         |    57500.000000 |
+----------+----------+------------+-----------------+
10 rows in set (0.00 sec)

mysql> select emp_name, salary,department,
    ->  sum(salary) over( order by salary, emp_id
    ->                    Rows between unbounded preceding and current row ) as dept_avg_salary
    ->  from employee;
+----------+----------+------------+-----------------+
| emp_name | salary   | department | dept_avg_salary |
+----------+----------+------------+-----------------+
| Meera    | 40000.00 | IT         |        40000.00 |
| Karan    | 45000.00 | Sales      |        85000.00 |
| Sneha    | 50000.00 | HR         |       135000.00 |
| Arjun    | 50000.00 | HR         |       185000.00 |
| Pooja    | 55000.00 | Sales      |       240000.00 |
| Vikram   | 55000.00 | Sales      |       295000.00 |
| Amit     | 60000.00 | IT         |       355000.00 |
| Neha     | 60000.00 | HR         |       415000.00 |
```

5. ROW_NUMBER()

Assigns a different sequential number to each row according to the window ordering.

```
SELECT emp_name, salary,
    ROW_NUMBER() OVER (
       ORDER BY salary DESC, emp_id
    ) AS row_num
FROM employee;
```

Theory

- Starts at 1 and gives every row a unique number.
- Tied salaries still receive different row numbers.
- Use a tie-breaker such as emp_id when you need deterministic numbering.

6. RANK()

Rows with the same value receive the same rank; the next rank is skipped after a tie.

```
SELECT emp_name, salary,
    RANK() OVER (
      ORDER BY salary DESC
    ) AS salary_rank
FROM employee
ORDER BY salary DESC, emp_id;
```

Theory

- The ORDER BY inside OVER determines rank assignment.
- The final ORDER BY controls display order; emp_id makes the order of tied rows predictable.
- If two rows rank 1, the next rank is 3.

```
mysql> select emp_name, salary,
    ->  row_number() over(order by salary, emp_id) as row_num
    ->  from employee;
+----------+----------+---------+
| emp_name | salary   | row_num |
+----------+----------+---------+
| Meera    | 40000.00 |       1 |
| Karan    | 45000.00 |       2 |
| Sneha    | 50000.00 |       3 |
| Arjun    | 50000.00 |       4 |
| Pooja    | 55000.00 |       5 |
| Vikram   | 55000.00 |       6 |
| Amit     | 60000.00 |       7 |
| Neha     | 60000.00 |       8 |
| Priya    | 80000.00 |       9 |
| Rahul    | 80000.00 |      10 |
+----------+----------+---------+
10 rows in set (0.00 sec)

mysql> select emp_name, salary,
    ->  Rank () over ( order by joining_date ) as rank_based_on_seniority
    ->  from employee;
+----------+----------+------------------------+
| emp_name | salary   | rank_based_on_seniority |
+----------+----------+------------------------+
| Neha     | 60000.00 |                      1 |
| Priya    | 80000.00 |                      2 |
| Vikram   | 55000.00 |                      3 |
| Amit     | 60000.00 |                      4 |
| Sneha    | 50000.00 |                      5 |
| Pooja    | 55000.00 |                      6 |
| Karan    | 45000.00 |                      7 |
| Rahul    | 80000.00 |                      8 |
| Arjun    | 50000.00 |                      9 |
```

```
mysql>  select emp_name, salary,
    ->  Rank () over ( order by joining_date ) as rank_based_on_seniority
    ->  from employee limit 1 ;
+----------+----------+-------------------------+
| emp_name | salary   | rank_based_on_seniority |
+----------+----------+-------------------------+
| Neha     | 60000.00 |                       1 |
+----------+----------+-------------------------+
1 row in set (0.00 sec)

mysql> select emp_name, salary,
    ->  Rank () over ( order by joining_date ) as rank_based_on_seniority
    ->  from employee limit 1 offset 1 ;
+----------+----------+-------------------------+
| emp_name | salary   | rank_based_on_seniority |
+----------+----------+-------------------------+
| Priya    | 80000.00 |                       2 |
+----------+----------+-------------------------+
1 row in set (0.00 sec)

mysql> select emp_name, salary,
    ->  Rank () over ( order by salary desc ) as rank_based_on_salary
    ->  from employee  ;
+----------+----------+---------------------+
| emp_name | salary   | rank_based_on_salary |
+----------+----------+---------------------+
| Priya    | 80000.00 |                   1 |
| Rahul    | 80000.00 |                   1 |
| Amit     | 60000.00 |                   3 |
| Neha     | 60000.00 |                   3 |
| Pooja    | 55000.00 |                   5 |
| Vikram   | 55000.00 |                   5 |
| Sneha    | 50000.00 |                   7 |
```

7. DENSE_RANK()

Like RANK(), tied values receive the same rank, but no rank numbers are skipped.

```
SELECT emp_name, salary,
      DENSE_RANK() OVER (
         ORDER BY salary DESC
      ) AS salary_rank
FROM employee
ORDER BY salary DESC, emp_id;
```

Theory

- Useful for ranking distinct salary values.
- If two rows rank 1, the next distinct salary receives rank 2.
- The final ORDER BY affects display only, not the rank calculation.

8. NTILE()

NTILE(n) distributes ordered rows into n buckets whose sizes differ by at most one row when possible.

```
SELECT emp_name, salary,
      NTILE(4) OVER (
         ORDER BY salary DESC, emp_id
      ) AS salary_bucket
FROM employee
ORDER BY salary DESC, emp_id;
```

Theory

- NTILE(4) creates four buckets.
- With ten rows, bucket sizes will be 3, 3, 2, and 2.
- Tied values can be split across buckets.

```
mysql> select emp_name, salary,
    ->  Dense_rank() over (order by salary desc ) as dense_rank_on_salary
    ->  from employee;
+----------+----------+----------------------+
| emp_name | salary   | dense_rank_on_salary |
+----------+----------+----------------------+
| Priya    | 80000.00 |                    1 |
| Rahul    | 80000.00 |                    1 |
| Amit     | 60000.00 |                    2 |
| Neha     | 60000.00 |                    2 |
| Pooja    | 55000.00 |                    3 |
| Vikram   | 55000.00 |                    3 |
| Sneha    | 50000.00 |                    4 |
| Arjun    | 50000.00 |                    4 |
| Karan    | 45000.00 |                    5 |
| Meera    | 40000.00 |                    6 |
+----------+----------+----------------------+
10 rows in set (0.00 sec)

mysql> select emp_name, salary,
    ->  Ntile(4) over ( order by salary desc, emp_id)  as salary_bucket
    ->  from employee;
+----------+----------+---------------+
| emp_name | salary   | salary_bucket |
+----------+----------+---------------+
| Priya    | 80000.00 |             1 |
| Rahul    | 80000.00 |             1 |
| Amit     | 60000.00 |             1 |
| Neha     | 60000.00 |             2 |
| Pooja    | 55000.00 |             2 |
| Vikram   | 55000.00 |             2 |
| Sneha    | 50000.00 |             3 |
| Arjun    | 50000.00 |             3 |
| Karan    | 45000.00 |             4 |
```

9. Aggregate window: SUM() OVER()

Calculates a total while keeping individual employee rows.

```
SELECT emp_name, department, salary,
    SUM(salary) OVER (
       PARTITION BY department
    ) AS dept_total_salary
FROM employee;
```

Theory

- Calculates the salary total for each department.
- Repeats the department total beside every employee in that department.
- Use GROUP BY instead when only one summary row per department is required.

```
mysql> select emp_name, salary,
    ->  Ntile(2) over ( order by salary desc, emp_id)  as salary_bucket
    ->  from employee;
+----------+----------+---------------+
| emp_name | salary   | salary_bucket |
+----------+----------+---------------+
| Priya    | 80000.00 |             1 |
| Rahul    | 80000.00 |             1 |
| Amit     | 60000.00 |             1 |
| Neha     | 60000.00 |             1 |
| Pooja    | 55000.00 |             1 |
| Vikram   | 55000.00 |             2 |
| Sneha    | 50000.00 |             2 |
| Arjun    | 50000.00 |             2 |
| Karan    | 45000.00 |             2 |
| Meera    | 40000.00 |             2 |
+----------+----------+---------------+
10 rows in set (0.00 sec)

mysql> select emp_name, salary,
    ->  Sum(salary) over () as total_salay
    ->  from employee;
+----------+----------+-------------+
| emp_name | salary   | total_salay |
+----------+----------+-------------+
| Amit     | 60000.00 |   575000.00 |
| Priya    | 80000.00 |   575000.00 |
| Rahul    | 80000.00 |   575000.00 |
| Sneha    | 50000.00 |   575000.00 |
| Neha     | 60000.00 |   575000.00 |
| Karan    | 45000.00 |   575000.00 |
| Pooja    | 55000.00 |   575000.00 |
| Vikram   | 55000.00 |   575000.00 |
```

10. Aggregate window: AVG() OVER()

Calculates an average alongside each row.

```
SELECT emp_name, department, salary,
    AVG(salary) OVER (
      PARTITION BY department
    ) AS dept_avg_salary
FROM employee;
```

Theory

- Shows the department's average salary next to every employee.
- Useful for comparing an employee's salary with their department average.

11. Aggregate window: COUNT() OVER()

Counts rows in each partition without collapsing the output.

```
SELECT emp_name, department,
    COUNT(*) OVER (
       PARTITION BY department
    ) AS employees_in_department
FROM employee;
```

Theory

- Displays the number of employees in the employee's department.
- Every employee row is preserved.

```
mysql> select emp_name, salary, department,
    ->  Sum(salary) over ( partition by department) as total_dept_salary
    ->  from employee;
+----------+----------+------------+--------------------+
| emp_name | salary   | department | total_dept_salary |
+----------+----------+------------+--------------------+
| Sneha    | 50000.00 | HR         |          160000.00 |
| Neha     | 60000.00 | HR         |          160000.00 |
| Arjun    | 50000.00 | HR         |          160000.00 |
| Amit     | 60000.00 | IT         |          260000.00 |
| Priya    | 80000.00 | IT         |          260000.00 |
| Rahul    | 80000.00 | IT         |          260000.00 |
| Meera    | 40000.00 | IT         |          260000.00 |
| Karan    | 45000.00 | Sales      |          155000.00 |
| Pooja    | 55000.00 | Sales      |          155000.00 |
| Vikram   | 55000.00 | Sales      |          155000.00 |
+----------+----------+------------+--------------------+
10 rows in set (0.00 sec)

mysql>
mysql>  select emp_name, department,
    ->  count(*) over (partition by department) as emp_per_department
    ->  from employee;
+----------+------------+--------------------+
| emp_name | department | emp_per_department |
+----------+------------+--------------------+
| Sneha    | HR         |                  3 |
| Neha     | HR         |                  3 |
| Arjun    | HR         |                  3 |
| Amit     | IT         |                  4 |
| Priya    | IT         |                  4 |
| Rahul    | IT         |                  4 |
| Meera    | IT         |                  4 |
| Karan    | Sales      |                  3 |
| Pooja    | Sales      |                  3 |
```

12. Running total

A running total adds the current value to all values earlier in the defined order.

```
SELECT emp_name, joining_date, salary,
      SUM(salary) OVER (
         ORDER BY joining_date, emp_id
         ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
      ) AS running_total
FROM employee
ORDER BY joining_date, emp_id;
```

Theory

- UNBOUNDED PRECEDING starts the frame at the first row.
- CURRENT ROW ends the frame at the current row.
- The total accumulates as rows are processed in joining-date order.

13. Moving average with ROWS BETWEEN

A moving average averages a limited number of ordered rows around the current row.

```
SELECT emp_name, joining_date, salary,
      sum (salary) OVER (
         ORDER BY joining_date, emp_id
         ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
      ) AS running_total
FROM employee
ORDER BY joining_date, emp_id;
```

Theory

- The frame includes up to two preceding rows and the current row: up to three rows.
- At the beginning, the frame contains fewer than three rows.
- This is a three-row window, not necessarily a three-day window.

```
mysql> select emp_name, salary,department,
    ->  sum(salary) over( order by salary, emp_id
    ->                   Rows between  2 preceding and current row ) as running_total
    ->  from employee;
+----------+----------+------------+---------------+
| emp_name | salary   | department | running_total |
+----------+----------+------------+---------------+
| Meera    | 40000.00 | IT         |      40000.00 |
| Karan    | 45000.00 | Sales      |      85000.00 |
| Sneha    | 50000.00 | HR         |     135000.00 |
| Arjun    | 50000.00 | HR         |     145000.00 |
| Pooja    | 55000.00 | Sales      |     155000.00 |
| Vikram   | 55000.00 | Sales      |     160000.00 |
| Amit     | 60000.00 | IT         |     170000.00 |
| Neha     | 60000.00 | HR         |     175000.00 |
| Priya    | 80000.00 | IT         |     200000.00 |
| Rahul    | 80000.00 | IT         |     220000.00 |
+----------+----------+------------+---------------+
10 rows in set (0.00 sec)
```

14. LAG() and LEAD()

LAG accesses a previous row's value; LEAD accesses a following row's value.

```
SELECT emp_name, joining_date, salary,
    LAG(salary) OVER (
        ORDER BY joining_date, emp_id
    ) AS previous_salary,
    LEAD(salary) OVER (
        ORDER BY joining_date, emp_id
    ) AS next_salary
FROM employee
ORDER BY joining_date, emp_id;
```

Theory

- The first row has no previous row, so LAG returns NULL by default.
- The last row has no next row, so LEAD returns NULL by default.
- The ORDER BY defines what previous and next mean.

```
mysql> select emp_name, joining_date , salary,
    ->  Lag(salary) over ( order by joining_date , emp_id ) as previous_salary,
    ->  Lead(salary) over ( order by joining_date, emp_id) as next_salary
    ->  from employee order by joining_date, emp_id;
+----------+--------------+----------+-----------------+-------------+
| emp_name | joining_date | salary   | previous_salary | next_salary |
+----------+--------------+----------+-----------------+-------------+
| Neha     | 2021-04-12   | 60000.00 |            NULL |    80000.00 |
| Priya    | 2021-06-15   | 80000.00 |        60000.00 |    55000.00 |
| Vikram   | 2021-09-22   | 55000.00 |        80000.00 |    60000.00 |
| Amit     | 2022-01-10   | 60000.00 |        55000.00 |    50000.00 |
| Sneha    | 2022-08-01   | 50000.00 |        60000.00 |    55000.00 |
| Pooja    | 2022-11-18   | 55000.00 |        50000.00 |    45000.00 |
| Karan    | 2023-01-05   | 45000.00 |        55000.00 |    80000.00 |
| Rahul    | 2023-03-20   | 80000.00 |        45000.00 |    50000.00 |
| Arjun    | 2023-07-09   | 50000.00 |        80000.00 |    40000.00 |
| Meera    | 2024-02-14   | 40000.00 |        50000.00 |        NULL |
+----------+--------------+----------+-----------------+-------------+
10 rows in set (0.00 sec)
```

Part 4 — Common Table Expressions (CTEs)

17. Basic CTE with WITH

A CTE is a named result set available to the SQL statement that follows it. It is not a permanent table.

```
WITH overall_average AS (
   SELECT AVG(salary) AS avg_salary
   FROM employee
)
SELECT e.emp_name, e.department, e.salary
FROM employee AS e
CROSS JOIN overall_average AS a
WHERE e.salary > a.avg_salary;
```

Theory

- The CTE calculates the overall average salary.
- The main query finds employees earning more than that average.
- A CTE helps make a multi-step query easier to read.

These approaches can solve similar problems. A CTE is not automatically faster; the main benefit is often clearer query structure.

```
mysql> With overall_average as (
    ->  select avg(salary) as avg_salary
    ->  from employee )
    ->  select e.emp_name, e.department, e.salary
    ->  from employee as e
    ->  cross join overall_average oa
    ->     On e.salary> oa.avg_salary ;
+----------+------------+----------+
| emp_name | department | salary   |
+----------+------------+----------+
| Amit     | IT         | 60000.00 |
| Priya    | IT         | 80000.00 |
| Rahul    | IT         | 80000.00 |
| Neha     | HR         | 60000.00 |
+----------+------------+----------+
4 rows in set (0.00 sec)
```

```
mysql> With overall_average as (
    ->  select avg(salary) as avg_salary
    ->  from employee )
    ->  select e.emp_name, e.department, e.salary
    ->  from employee as e
    ->  cross join overall_average oa
    ->     On e.salary <oa.avg_salary ;
+----------+------------+----------+
| emp_name | department | salary   |
+----------+------------+----------+
| Sneha    | HR         | 50000.00 |
| Karan    | Sales      | 45000.00 |
| Pooja    | Sales      | 55000.00 |
| Vikram   | Sales      | 55000.00 |
| Meera    | IT         | 40000.00 |
| Arjun    | HR         | 50000.00 |
+----------+------------+----------+
6 rows in set (0.00 sec)
```

19. Chained CTEs

Multiple CTEs can be defined in one WITH clause. A later CTE can refer to an earlier CTE.

```
WITH department_averages AS (
   SELECT department, AVG(salary) AS avg_salary
   FROM employee
   GROUP BY department
),
```

```
above_average AS (
   SELECT e.emp_name, e.department, e.salary, d.avg_salary
   FROM employee AS e
   JOIN department_averages AS d
     ON e.department = d.department
   WHERE e.salary > d.avg_salary
)
SELECT *
FROM above_average
ORDER BY department, salary DESC;
```

Theory

- The first CTE calculates the average salary per department.
- The second CTE finds employees above their department's average.
- The final SELECT displays the second CTE's result.

```
mysql> with department_avg as (
    -> select department, avg(
    -> salary) as avg_salary
    -> from employee
    -> group by department),
    -> above_avg as ( select e.emp_name, d.department, e.salary, d.avg_salary
    -> from employee as e
    -> Join department_avg  as d
    -> On e.department = d.department
    -> where e.salary > d.avg_salary
    -> )
    -> select * from above_avg order by department, salary desc;
+----------+------------+----------+--------------+
| emp_name | department | salary   | avg_salary   |
+----------+------------+----------+--------------+
| Neha     | HR         | 60000.00 | 53333.333333 |
| Priya    | IT         | 80000.00 | 65000.000000 |
| Rahul    | IT         | 80000.00 | 65000.000000 |
| Pooja    | Sales      | 55000.00 | 51666.666667 |
| Vikram   | Sales      | 55000.00 | 51666.666667 |
+----------+------------+----------+--------------+
5 rows in set (0.00 sec)
```

20. Recursive CTE theory

A recursive CTE refers to itself. It is useful for generating sequences and traversing parent-child structures such as reporting hierarchies.

- Anchor member: produces the starting row(s) and does not refer to the CTE itself.
- Recursive member: refers to the CTE to produce the next row(s).
- Termination: recursion stops when the recursive member produces no more rows or the recursion limit is reached.
- The anchor and recursive member are commonly joined with UNION ALL.

- MySQL supports recursive CTEs with WITH RECURSIVE (MySQL 8.0+).

21. Recursive CTE — number series

Generate numbers from 1 through 10 without creating a permanent table.

```
WITH RECURSIVE numbers AS (
  -- Anchor member
  SELECT 1 AS n

  UNION ALL

  -- Recursive member
  SELECT n + 1
  FROM numbers
  WHERE n < 10
)
SELECT n
FROM numbers;
```

Theory

- The anchor starts with 1.
- The recursive member adds 1 to the previous result.
- When n reaches 10, n < 10 is false and no more rows are generated.
- Always provide a valid stopping condition. MySQL normally has a default CTE recursion-depth limit of 1000.

```
mysql> with recursive numbers as (
    ->
    -> select 1 as n  -- n=1
    ->
    -> union all
    ->
    -> select n+1 -- n+1,2, 3,4,5,6,7,8,9,10
    -> from numbers
    -> where n<10
    -> )
    -> select n from numbers;
+------+
| n    |
+------+
|    1 |
|    2 |
|    3 |
|    4 |
|    5 |
|    6 |
|    7 |
|    8 |
|    9 |
|   10 |
+------+
10 rows in set (0.00 sec)
```

Topic	Remember
GROUP BY	Summarizes rows; normally returns one row per group.
OVER()	Defines a window and preserves row-level output.
PARTITION BY	Starts a separate calculation window for each group.
ROW_NUMBER()	Unique sequential number for each row.
RANK()	Ties share ranks; later ranks have gaps.
DENSE_RANK()	Ties share ranks; later ranks have no gaps.
NTILE(n)	Divides ordered rows into approximately even buckets.
ROWS BETWEEN	Defines the row frame used by a window calculation.
LAG / LEAD	Reads previous / next row values.
CTE	A named result set defined using WITH for one statement.
Recursive CTE	Anchor + recursive member + termination condition.